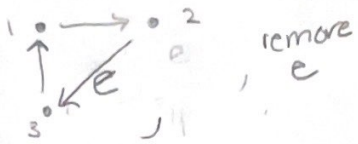
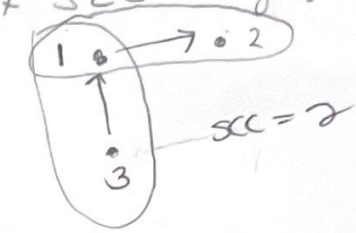
Estela Bobadilla-Cruz

Q1

1a) G is directed and has only one cycle. Let e be an edge on that cycle; we remove e from G but keep the end vertices of e in G. Does # of SCC change?

Let G =





SCC = 1

yes the number of SCC will increase by 1 because at first G only had one strong CC, but after removing edge e that connects two vertices that were a part of our SCC and reachable from each other, we know these two vertices (in this example 2, 3) are no longer reachable from eachother. Therefore we now have two SCC, each with one end of the vertices of edge e.

1b) Given: Unairected connected graph G that is also weighted.

Prims alg considers edges at the graph by increasing order of weight (so taking the min w(u,v)). And it constructs the MST by adding the min weight edge that connects a vertex in the MST to a vertex that has not been chosen yet (Not in our MST). When we square the weights of the edges, the order in which the edges are considered will also change so the min weight edge chosen at each step to build our MST may also change (could be different order by which edges were chosen before squaring the weights) CONTINUED →

Q1

1b) The order in which the edges are chosen in Prims Alg may be different, but it also can be the same, though that is Not always guaranteed.

Estela Cruz

Q2

Given: n square size letters and m square-size envelopes.

Each letter has a length, each envelope has a length

P[1...n] P[i] denotes len of paper

E [1...m] len of envelopes

*1p in 1e, can only put inside if len p < len e

sort this way b/c P[i] < e[i]

Greedy Approach:

We would first sort the papers by increasing order of len (ascending) and sort the envelopes in decreasing order (descending). For each P[i] in P[n] we will try to put the paper inside the first envelope that is largest (Since our envelope array goes from big → small len)

If we can successfully place the paper into our envelope will be mark this e[i] as "used" and remove it from the list of "Not used" envelopes

If the paper does not fit, that is, P[i] > e[i], we will continue our alg and move onto the next paper.

This greedy strategy essentially always tries to place the smallest "available/Not used" paper into the largest "Not used" envelope to try to use as many envelopes as we can because our goal is to to put as many papers in envelopes. We know that to place P[i] (paper) into e[i], P[i] < e[i] so if a paper fits, into envelope then all larger envelopes can also fit this paper. Our alg sorts e in descending order so we consider the largest envelopes first for maximum results

using sorting alg, we can say TC = O(nlogn)

Evan Banz Q3

given: Sue is given $G=(V,E)$
Sue labels good/bad
Wants cycle w/ u with least amount of bad edges.

Sue can run DFS(G, u) given source vertex u and find all the vertices reachable from u. We store/ Place these vertices in Q. then for each vertex in Q (Q[v]) we can run DFS and see how many bad edges there are when we run DFS(Q[v]). If the results from DFS return our source vertex u, then we know a cycle has been found that includes u. We will then update then number of bad edges if necessary. DFS takes $O(|E|+|V|)$ and we find Q by using DFS that considers vertices only once so that will take $O(|V| \log|V|)$. Here, we know that we can find a cycle by simply using DFS and ensuring that DFS only visits vertices once.

(Q4)

4a) G is undirected connected graph, let V be a vertex of G
Bob runs BFS (G,V), records Num layers
Bob runs DFS (G,V), records Num layers

Yes, Bob is correct because BFS explores all the vertices "layer by layer" or by "level" before moving onto the next layer at the G. So it'll start from source vertex of G and visit all of its neighbors BEFORE moving onto the next layer of neighboring vertices whereas DFS goes as far as possible along a branch first.

4b) G(V,E) is directed Acyclic Graph (DAG). Input = source vertex G, and pos Number K. Determine if there is a path in G from s consisting at least k vertices. TC = O(|V|+|E|)

Our algorithm will keep a counter variable to count how many vertices we have added to our path. We can store all of our unused vertices in a stack While(counter < K) we will pop a vertex from our stack G with unused vertices and add it to our path and increase our counter. We do have to have an updating rule where we "push"/add all the neighbors of V that have NOT been visited. So that we only visit each vertex and each edge once, so TC = O(|V|+|E|)

Q5

Estela

Sue can use BFS, first we can have a queue Q with vertex s. We can perform BFS(s) and at each vertex v, we can check if v = t, if it is we can simply return True, if we explored the whole graph and have not visited t (can have key values like t.key = visited or keep a bool to see if t is visited) then we would return false

We can store the number of good edges used to reach each vertex and check if it is even or odd. We also have to remove a vertex v from Q and add all neighboring vertices that have been unvisited. We will only visit a vertex / edge once. to ensure TC = $O(|V|+|E|)$